

Docentes: Ricardo Cerri (cerri@icmc.usp.br)
Diego Furtado Silva (diegofsilva@icmc.usp.br)

Pessoas Monitoras: João Pedro Conde Gomes Alves
Bruno Uhlmann Marcato
Arthur Alexandre Santos Santana
Gabriel Cordeiro Correa Silva

Cadastros

Autor: João Pedro Conde

1 Descrição

Chegou o grande dia: a OBI (Olimpiada Brasileira Interessante) abriu as inscrições! Muitos do ICMC vão participar e, para organizar as inscrições desse incrível instituto, foi planejado fazer um sistema de gerenciamento.

Nesse sistema, é possível fazer duas operações básicas de gerência:

- **Cadastro [1]:** adiciona uma nova pessoa no sistema;
- **Remoção [2]:** retira a pessoa mais recentemente adicionada no sistema que ainda não tenha sido removida.

O armazenamento desse sistema é dado por um vetor de strings, em que cada posição guarda o nome de uma pessoa cadastrada.

Como o ICMC preza pela eficiência, o instituto rejeita soluções em que deixa o vetor seja grande demais para poucos dados. Portanto, ele deseja que o vetor cresça conforme o número de cadastros cresça e diminua conforme o número de remoções. Ou seja, é necessário um vetor alocado dinamicamente!

Mas, o uso do realloc a cada cadastro e remoção seria custoso demais... Para contornar isso, existe a seguinte estratégia: quando necessário aumentar o vetor, dobra-se seu tamanho; quando é possível diminuí-lo, divide seu tamanho atual por 2. Assim, o tamanho assume somente potências de 2 como valor.

Por exemplo, o tamanho `tam` do vetor de armazenamento do sistema, inicialmente, é 1 e supomos que essa posição única é ocupada pelo nome "João", como a seguir:

João

Ao cadastrar o nome "Maria", será necessário aumentar o tamanho do vetor, pois a única posição alocada está ocupada. Desse modo, conforme a estratégia especificada, realoca-se o vetor dobrando seu tamanho, isto é, $\text{tam} = 2 * \text{tam}$.

João	Maria
------	-------

Se ainda for cadastrado outra pessoa - por exemplo, "Mateus" -, `tam` deve ser aumentado de novo, pois não há espaço para um novo cadastro (as duas posições do vetor estão ocupadas). Assim, `tam = 2 * tam` novamente, resultando em `tam` igual a 4.

João	Maria	Mateus	-
------	-------	--------	---

Perceba que apesar de ter sido alocado 4 posições, somente 3 estão devidamente ocupadas. Nesse caso, então, seria possível adicionar mais uma pessoa sem realizar outro `realloc`, sendo essa a vantagem dessa estratégia.

Por outro lado, quando o vetor possuir somente metades dos espaços alocados sendo ocupados, divide-se o tamanho do vetor por 2. Voltando no exemplo supracitado, se agora fosse realizado a operação de remoção, "Mateus" sairia do vetor (a última pessoa cadastrada não removida) e, somente metade dos espaços alocados seriam necessários, sendo possível assim realocar o vetor com seu tamanho pela metade.

João	Maria
------	-------

Viu que legal? Essa estratégia é tão legal que foi aprovada pelo ICMC para a construção do sistema.

Ademais, lembre: o ICMC é eficiente. Desse modo, a alocação das strings também devem ser bem feitas: a string em cada posição do vetor deve ter somente o tamanho necessário. Ou seja, na primeira posição do armazenamento ilustrado acima, o espaço alocado para guardar o nome "João" deve ter exatamente 5 bytes (um para cada caractere do nome e um para o terminador de string).

Outrossim, existe uma terceira operação que deve ter nesse sistema:

- **Relatório [3]:** retorna a quantidade B de bytes alocados no armazenamento.

B inclui tanto o espaço alocado para ponteiros no vetor quanto o espaço alocado para cada string. Contudo, não inclui o próprio ponteiro para o vetor de strings. Voltando mais uma vez no exemplo, no último cenário, essa operação deveria retornar 27 (5 bytes para armazenar "João", 6 bytes para armazenar "Maria" e 16 bytes para armazenar os ponteiros das duas posições do vetor).

Enfim, você - o honrado do ICMC - foi chamado para fazer esse sistema.

2 Instruções Complementares

- A primeira linha de entrada contém $1 \leq N \leq 10^3$ o número de operações a serem executadas.
- As próximas N linhas descrevem as operações. Cada uma delas se inicia com um inteiro p que indica a operação a ser realizada (1 para Cadastro, 2 para Remoção e 3 para Relatório). Para a operação de Cadastro ainda, após um espaço, é dado o nome s da pessoa a ser cadastrada - $1 \leq |s| \leq 30$.

- A saída é composta por N linhas, cada uma referente a uma operação (na ordem em que foram solicitadas):
 - **Cadastro:** deve imprimir "Realocacao", se foi necessário aumentar o tamanho do vetor; caso contrário, "-".
 - **Remoção:** deve imprimir o nome da pessoa removida e "Realocacao", se o vetor diminuiu de tamanho; caso contrário, "-" (o nome é impresso em qualquer um dos dois casos).
 - **Relatório:** deve imprimir a quantidade B de bytes alocados pelo sistema (perceba que é **alocados** e não de fato ocupados).
- Coloque uma quebra de linha no final da saída.
- No mínimo, o tamanho do vetor é 1 (não é possível realocar com tamanho 0).
- Caso seja chamada a instrução de remoção com o vetor vazio, deve-se somente imprimir "-", sem modificar o vetor.
- **É obrigatório o uso de alocação dinâmica e realocação seguindo a estratégia exposta. Lembre de limpar a memória.**
- Modularize seu código.
- **Dica:** para leitura das strings, use um vetor de char estático de 31 posições e, depois, copie o conteúdo do vetor para o local em que foi alocado somente o necessário.
- Submeta o arquivo .c no sistema: <https://runcodes.icmc.usp.br>
- **Atenção:** O *Run Codes* é sensível a espaços, pontos e quebras de linha. A saída deve ser **idêntica** à esperada.

3 Exemplos de Entrada e Saída

Entrada

```
7
1 Joao
1 Maria
3
2
1 Lucca
1 Alexandre
3
```

Saída

```
-
Realocacao
27
Maria Realocacao
Realocacao
Realocacao
53
```

Entrada

```
7
1 Joao
1 Mateus
1 Lucas
1 Filipe
1 Mateus
1 Ana
3
```

Saída

```
-
Realocacao
Realocacao
-
Realocacao
-
100
```